

K8s - Installation Using Kind and microk8s and minikube

#Install k8s with kind on windows

✓ Step-by-Step Instructions

1. 🔄 Move **kind.exe** to a permanent location

1. Download and rename the binary:
 - Rename **kind-windows-amd64** to **kind.exe**

Move it to a folder like:

C:\Program Files\kind\

2.
 - If **C:\Program Files\kind** doesn't exist, create it.
 - You might need admin rights to move the file there.

2. ⚙️ Add the folder to your System PATH

🪟 For Windows 10/11:

1. Press **Win + S**, type **"Environment Variables"**, and click:

"Edit the system environment variables"

2. In the **System Properties** window:
 - Click on the “**Environment Variables...**” button.
3. Under **System variables**, scroll down and select **Path**, then click **Edit**.
4. In the **Edit Environment Variable** window:

Click **New**, then enter:

C:\Program Files\kind\

-
5. Click **OK** on all open dialogs to save.

3. Restart your terminal

Close and reopen PowerShell or CMD.

4. Verify

Run:

kind version

You should see output like:

kind v0.22.0

#Install Kind On Ubuntu

To install **Kind** (Kubernetes IN Docker) on **Ubuntu**, follow these clean and reliable steps:

Prerequisites

1. Docker must be installed and running

To install Docker on Ubuntu:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
```

Add your user to the Docker group (optional, to avoid using **sudo**):

```
sudo usermod -aG docker $USER
```

 **Note:** You must **log out and log back in** (or reboot) after this to apply the group change.

Install Kind on Ubuntu

Step 1: Download the Kind binary

```
curl -Lo kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
```

Step 2: Make it executable

```
chmod +x kind
```

Step 3: Move it to your PATH

```
sudo mv kind /usr/local/bin/kind
```

Verify the installation

Run:

```
kind version
```

Expected output:

```
kind v0.22.0
```

Create your first cluster

```
kind create cluster
```

This will spin up a Kubernetes cluster using Docker containers.

Let me know if you also want to:

- Delete a cluster
- Use a custom config
- Integrate Kind with `kubectl` or Helm

Kubernetes Installation with KOPS

What is kops?

kops (short for **Kubernetes Operations**) is an open-source tool that helps you **install, upgrade, and manage Kubernetes clusters** in the cloud or on-premises. It's often described as `kubectl` for cluster operations.

What Can You Do with kops?

- Create, configure, and launch **production-grade Kubernetes clusters**
 - Manage Kubernetes **cluster lifecycle** (create, update, delete)
 - Automatically provision **infrastructure** (VMs, networking, IAM, etc.)
 - Maintain **state** in an S3 bucket or Git repo
 - Supports **high availability** (HA) setups
-

Where Can kops Be Used?

Primarily for cloud environments:

-  **AWS** (most mature support)
 -  Also supports:
 - GCP
 - DigitalOcean
 - OpenStack
 - VMware (beta)
 - Bare metal (with some extra setup)
-

How Does kops Work?

1. **You define your cluster** in a manifest (YAML or command line).
2. **kops uses that configuration** to generate all the cloud infrastructure (via Terraform or directly).
3. It installs Kubernetes on the provisioned nodes.
4. You then use **kubectl** to interact with the cluster.

#Create a ubuntu vm
#With t2.micro for work-station

#Install and configure aws cli

To install the AWS Command Line Interface (CLI) on Ubuntu, follow these steps:

✅ **Step 1: Update your package list**

`sudo apt update`

✅ **Step 2: Install required dependencies**

`sudo apt install unzip curl -y`

✅ **Step 3: Download the AWS CLI installer**

`curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`

💡 For ARM-based systems (e.g., Raspberry Pi), use this URL instead:

<https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip>

✅ **Step 4: Unzip the installer**

`unzip awscliv2.zip`

✅ **Step 5: Run the installer**

```
sudo ./aws/install
```

✅ Step 6: Verify installation

```
aws --version
```

You should see output like:

```
aws-cli/2.x.x Python/3.x.x Linux/x86_64
```

✅ Optional: Remove installation files

```
rm -rf awscli2.zip aws/
```

#Install and Configure KOPS

To install **kops** on Ubuntu, follow these steps:

🧰 Step-by-Step Installation of kops on Ubuntu

✅ Step 1: Install **curl** (if not installed)

```
sudo apt update
```

```
sudo apt install curl -y
```

✅ Step 2: Download the latest **kops** binary

`curl -LO`

`https://github.com/kubernetes/kops/releases/latest/download/kops-linux-amd64`

💡 For ARM-based systems (e.g., Raspberry Pi), replace `kops-linux-amd64` with `kops-linux-arm64`.

✅ Step 3: Make the binary executable

`chmod +x kops-linux-amd64`

✅ Step 4: Move it to a system directory

`sudo mv kops-linux-amd64 /usr/local/bin/kops`

✅ Step 5: Verify the installation

`kops version`

You should see output like:

Version 1.xx.x (git-xxxxxxxxx)

✅ Optional: Enable bash completion

`kops completion bash | sudo tee /etc/bash_completion.d/kops > /dev/null`

`source /etc/bash_completion`

#Install and Configure Kubectl

What is **kubectl**?

kubectl is the command-line tool used to interact with **Kubernetes clusters**. It allows you to deploy applications, inspect resources, and manage the cluster itself.

Common Uses of **kubectl**

- **View** cluster resources (pods, services, deployments, nodes, etc.)
- **Create, update, and delete** Kubernetes objects
- **Execute commands** inside running containers
- **Apply configuration** files (YAML/JSON)
- **Debug** workloads with logs, exec, describe, etc.

Here's how to **install kubectl** (the Kubernetes CLI) on **Ubuntu**:

Step-by-Step: Install **kubectl** on Ubuntu

♦ Step 1: Download the latest **kubectl** binary

```
curl -LO "https://dl.k8s.io/release/$(curl -Ls  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

 For ARM systems (like Raspberry Pi), change **amd64** to **arm64**.

♦ **Step 2: Make it executable**

```
chmod +x kubectl
```

♦ **Step 3: Move it to a system path**

```
sudo mv kubectl /usr/local/bin/
```

♦ **Step 4: Verify the installation**

```
kubectl version --client
```

Expected output:

Client Version: v1.xx.x

✅ **Optional: Enable bash autocompletion**

```
sudo apt install bash-completion -y
```

```
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl > /dev/null
```

```
source /etc/bash_completion
```

#Create and Configure S3 Bucket

To **create an S3 bucket for `kops`** and **export the `KOPS_STATE_STORE`** environment variable, follow these steps:

Why is this needed?

`kops` uses **S3 to store the cluster state**, including configuration, secrets, and manifests. You must set this up before creating a cluster.

Step-by-Step: Create S3 Bucket & Set `KOPS_STATE_STORE`

1. Choose a unique bucket name

S3 bucket names must be **globally unique**. Use a name like:

```
export KOPS_BUCKET_NAME=my-kops-state-$(date +%s)
```

2. Create the S3 bucket

Replace `us-west-2` with your preferred AWS region:

```
aws s3api create-bucket \  
  --bucket $KOPS_BUCKET_NAME \  
  --region us-west-2 \  
  --create-bucket-configuration LocationConstraint=us-west-2
```

3. Enable versioning on the bucket

Versioning is recommended so `kops` can safely track changes:

```
aws s3api put-bucket-versioning \  
  --bucket $KOPS_BUCKET_NAME \  
  --versioning-configuration Status=Enabled
```

4. Export the **KOPS_STATE_STORE** variable

```
export KOPS_STATE_STORE=s3://$KOPS_BUCKET_NAME
```

You can add this to your shell config (`~/.bashrc`, `~/.zshrc`) to make it persistent:

```
echo "export KOPS_STATE_STORE=s3://$KOPS_BUCKET_NAME" >> ~/.bashrc  
source ~/.bashrc
```

✅ Done!

#Create a Kubernetes Two-Node Cluster

```
kops create cluster --name=test.k8s.local --zones=us-west-1a,us-west-1b  
--node-count=2  
--state=s3://kops-aja-demo --yes
```

#This will create a two-node cluster

Below are **clean, structured teaching notes** on **Modifying & Updating Instance Groups in kOps**, written for **DevOps / CKA / Kubernetes training** purposes.

You can directly use this in **class notes, lab docs, or internal wiki** 

 **kOps – Modify & Update Instance Groups (IGs)**

♦ What is an Instance Group in kOps?

An **Instance Group (IG)** in kOps defines **how Kubernetes nodes are created and managed**.

It controls:

- Number of nodes
- Instance type
- Disk size
- Autoscaling limits
- Labels & taints
- Spot / On-Demand settings

Each IG maps to an **Auto Scaling Group (ASG)** in AWS.

♦ Types of Instance Groups

Type	Purpose
Node	Worker nodes
Master	Control plane nodes
Bastion	SSH access
APIServer	API server only

♦ When Do We Modify Instance Groups?

You modify IGs when you want to:

- ✓ Change instance type (t3 → t3.large)
 - ✓ Enable or update autoscaling
 - ✓ Increase disk size
 - ✓ Add node labels / taints
 - ✓ Switch to Spot instances
 - ✓ Change AMI or OS
 - ✓ Tune performance or cost
-

♦ Step 1: List Instance Groups

kops get ig --name <cluster-name>

Example:

kops get ig --name demo.k8s.local

📌 Output shows all IGs like:

- master-us-east-1a
 - nodes
-

♦ Step 2: Modify Instance Group

✎ Method 1: Edit Directly (Most Common)

kops edit ig <ig-name> --name <cluster-name>

Example:

kops edit ig nodes --name demo.k8s.local

➡ Opens YAML in editor (vim/nano)

Method 2: Export → Edit → Replace (Recommended for Production)

```
kops get ig nodes --name demo.k8s.local -o yaml > nodes-ig.yaml
```

Edit the file, then:

```
kops replace -f nodes-ig.yaml
```

◆ **Common Fields to Modify (IMPORTANT)**

◆ **Enable Auto Scaling**

spec:

minSize: 2

maxSize: 6

 Enables ASG-based scaling

◆ **Change Instance Type**

spec:

machineType: t3.large

◆ **Increase Disk Size**

spec:

volumeSize: 100

volumeType: gp3

◆ Add Node Labels

spec:

labels:

role: backend

env: prod

◆ Add Node Taints

spec:

taints:

- key=dedicated

value=gpu

effect=NoSchedule

◆ Use Spot Instances

spec:

maxPrice: "0.05"

📌 Converts nodes to Spot instances

◆ Step 3: Preview Changes (VERY IMPORTANT)

kops update cluster --name <cluster-name> --dry-run

📌 Shows what will change **without applying**

♦ Step 4: Apply the Changes

kops update cluster --name <cluster-name> --yes

 Updates ASG / Launch Templates

♦ Step 5: Rolling Update (When Required)

Some changes require **node recreation**:

- ✓ Instance type
- ✓ Disk size
- ✓ AMI
- ✓ Networking

Run:

kops rolling-update cluster --name <cluster-name> --yes

 Nodes are drained and recreated safely

♦ Full Example: Modify Worker Nodes

Goal:

- Instance type → **t3.large**
- Autoscaling → **2-5**
- Disk → **80GB**

spec:

machineType: t3.large

minSize: 2
maxSize: 5
volumeSize: 80
labels:
 role: worker

Commands:

```
kops edit ig nodes --name demo.k8s.local  
kops update cluster --name demo.k8s.local --yes  
kops rolling-update cluster --name demo.k8s.local --yes
```

◆ Best Practices ★

- ✓ Always run `--dry-run` first
 - ✓ Use **rolling-update** for infra changes
 - ✓ Keep separate IGs for different workloads
 - ✓ Use labels + taints for workload isolation
 - ✓ Do not change master IG casually
 - ✓ Monitor cluster during rolling updates
-